

Segmentation des points chauds sur les panneaux solaires

Préparé par : Groupe 10

El Aoide Amal 23

Hanine Sara 36

Amraoui Chaimae 6

El Ansari Douae 22

El Kachtoul Salma 29

Encadrants :

Mme SEBARI IMANE

Mme AIT EL KADI KENZA

Monsieur Jabrane Mouad

REMERCIEMENT

Nous tenons à exprimer notre profonde reconnaissance à toutes les personnes qui ont contribué, de manière directe ou indirecte, à la réalisation de ce projet.

Nous remercions tout particulièrement notre encadrant, Monsieur Jabrane Mouad pour sa disponibilité, sa bienveillance, et la qualité de son accompagnement tout au long de ce travail. Son expertise technique, ses conseils pédagogiques et ses retours constructifs ont été déterminants pour structurer notre démarche et mener à bien notre projet dans les meilleures conditions. Nous avons pu bénéficier d'un encadrement rigoureux, tout en conservant une certaine autonomie dans la gestion des tâches, ce qui a été très formateur.

Nos remerciements s'adressent également à Mme. Sebari Imane et Mme. Ait El Kadi Kenza pour le cadre de travail fourni .

Enfin, nous remercions sincèrement tous les membres de notre groupe pour leur engagement, leur sérieux et leur esprit de collaboration. La réussite de ce projet est le fruit d'un travail collectif fondé sur la complémentarité de nos compétences, la rigueur dans l'organisation, et une volonté commune d'explorer les enjeux technologiques liés à la détection automatisée des points chauds sur panneaux solaires.

Table des matières

I.	INTRODUCTION :	5
I.1	Définition d'un point chaud sur les panneaux solaires :	5
I.2	Causes et risques des points chauds :	5
I.2.1	Causes d'apparition des points chauds :	5
I.2.2	Les risques liés aux points chauds pour les panneaux solaires :	5
II.	Approche méthodologique de traitement sous ERDAS Imagine :	6
II.1	Données et Matériel :	6
II.1.1	Données satellitaires d'entrée (Sentinel-2 et Landsat 8) :	6
II.1.2	Sources des données :	6
II.2	Logiciels et Méthodologie :	7
II.2.1	Description de logiciel Erdas :	7
II.2.2	Limites d'ERDAS Imagine 2011	7
II.2.3	Complémentarité avec QGIS	8
II.2.4	Méthodologie générale adoptée	8
II.3	Prétraitement et contraintes :	8
II.4	Traitement de l'image multispectrale :	10
II.4.1	Application d'indices :	10
II.4.2	Détection des panneaux solaires (classification supervisée)	10
II.5	Détection des points chauds (analyse thermique)	12
II.6	Fusion des résultats et contraintes	13
III.	Approche méthodologique de traitement en utilisant le deep-learning:	14
III.1	Définition du Deep Learning :	14
III.2	U-Net :	14
III.3	Contraintes et limites de l'approche par U-Net :	15
III.3.1	Contraintes liées aux données d'entrée :	15
III.3.2	Contraintes liées aux limites naturelles du modèle U-Net :	15
III.4	Données d'entrées utilisées :	15
III.5	Les outils et bibliothèques utiles	16
III.6	Chargement des images et annotations	17
III.7	Le but d'entraînement du modèle U-Net :	17
III.8	Construction et entraînement du modèle U-Net	17
IV.	Segmentation des points chauds sur les panneaux solaires Par deeplearning	20
IV.1	Etapes :	20
IV.1.1	Entraînement :	20

IV.1.2	Affichage :	24
IV.1.3	Problème rencontré dans la partie d'entraînement :	25
IV.1.4	Solutions :	25
IV.1.5	Evaluation du modèle :	27
IV.1.6	Visualisation :	32
IV.1.7	Analyse quantitative :	34
IV.1.8	Comparaison entre le masque réel et le masque predict:	36
IV.1.9	Problèmes rencontrés :	43
IV.2	Conclusion :	44

Liste des figures :

Figure 1: zone choisie	7
Figure 2: Extrait du fichier MTL	8
Figure 3: conversion DN/Celsius.....	9
Figure 4:Resultat :layer stack.....	9
Figure 5: Résultat du NDVI.....	10
Figure 6: Valeur de l'indice au niveau d'un pixel du panneau	10
Figure 7:Resultat de classification supervisée = mauvais résultat	12
Figure 8: exemple d'image de Dataset_2	15
Figure 9: exemple d'image de Dataset_1	15
Figure 10; résultat de l'entraînement	24

I. INTRODUCTION :

I.1 Définition d'un point chaud sur les panneaux solaires :

Les points chauds sont des zones de température élevée qui n'affectent qu'une seule zone du panneau solaire et entraînent une diminution localisée de l'efficacité et, par conséquent au lieu de produire de l'électricité comme prévu, se met à dissiper l'énergie sous forme de chaleur. Les panneaux solaires génèrent de l'énergie et des points chauds peuvent survenir lorsque, pour une série de causes que nous énumérerons, une partie de cette énergie se dissipe, au lieu d'être générée, dans une zone localisée, un peu comme si les cellules consumaient l'énergie au lieu de la produire. De plus, les points chauds sont généralement instables et s'intensifient généralement jusqu'à ce que la performance totale du panneau soit nulle.

I.2 Causes et risques des points chauds :

I.2.1 Causes d'apparition des points chauds :

➤ Ombres partielles et obstacles :

Les branches d'arbres, cheminées, feuilles mortes ou dépôts de poussière peuvent créer des ombres sur certaines cellules, réduisant ainsi leur capacité à produire de l'électricité et favorisant l'échauffement.

➤ Défauts de fabrication ou cellules abîmées :

Certains panneaux solaires présentent des défauts internes dès leur fabrication ou se dégradent avec le temps, augmentant ainsi la probabilité d'apparition des points chauds.

➤ Vieillissement des panneaux

Avec les années, l'exposition aux intempéries et aux variations de température peut provoquer des microfissures dans les cellules photovoltaïques, ce qui altère leur efficacité.

➤ Mauvais montage ou connexion défectueuse

Un câblage mal conçu ou des connexions mal réalisées peuvent provoquer des points chauds en raison de pertes de charge excessives.

➤ Saleté et sable :

Les panneaux photovoltaïques peuvent se salir en raison de la poussière, du sable en suspension, de la saleté et d'autres impuretés contaminantes pendant leur durée de vie.

I.2.2 Les risques liés aux points chauds pour les panneaux solaires :

Un point chaud peut avoir plusieurs conséquences néfastes

➤ Baisse du rendement : une cellule affectée limite la production d'énergie de l'ensemble du panneau.

➤ Risque d'incendie : dans des cas extrêmes, la surchauffe peut endommager l'isolant et créer des étincelles.

- Risque de panne totale : Si le point chaud devient trop intense, il peut rendre le panneau ou la chaîne de panneaux inopérants.

II. Approche méthodologique de traitement sous ERDAS Imagine :

II.1 Données et Matériel :

II.1.1 Données satellitaires d'entrée (Sentinel-2 et Landsat 8) :

Notre approche combine deux types d'images satellites : (1) une image Sentinel-2 MSI à haute résolution (Bandes 2, 3, 4, 8) pour localiser visuellement les panneaux solaires ; (2) une image Landsat 8 TIRS (Thermal Infrared Sensor, bande 10) à résolution plus grossière pour détecter les points chauds thermiques. Les bandes VNIR de Sentinel-2 (bleu, vert, rouge et proche infrarouge) sont acquises à 10 m de résolution spatiale, ce qui permet de distinguer les objets (panneaux) de petite taille. En revanche, la bande 10 de Landsat 8 (10.6–11.2 μm , thermique) a une résolution native d'environ 100 m (re-échantillonnée à 30 m dans les produits finaux). Ainsi, l'image Sentinel-2 comporte beaucoup plus de détails spatiaux qu'une image thermique Landsat.

II.1.2 Sources des données :

Les données satellitaires utilisées dans ce projet proviennent de deux plateformes ouvertes et reconnues :

- **EO Browser (Sentinel Hub)** : cette plateforme permet la visualisation, la sélection et le téléchargement d'images Sentinel-2 à haute résolution. Les bandes B2 (bleu), B3 (vert), B4 (rouge) et B8 (proche infrarouge) ont été extraites au format GeoTIFF. EO Browser offre l'avantage de fournir des données prétraitées (réflectance de surface), prêtes à l'analyse.
- **USGS Earth Explorer** : ce portail fournit un accès aux images Landsat, notamment à la **bande thermique B10** de Landsat 8, essentielle pour la détection des anomalies thermiques. Les métadonnées associées (fichier MTL) ont également été téléchargées pour effectuer les conversions nécessaires (DN $\rightarrow$ température).



Figure 1: zone choisie

II.2 Logiciels et Méthodologie :

II.2.1 Description de logiciel Erdas :

ERDAS Imagine est un logiciel classique de traitement d'images satellitaires, riche en outils d'analyse spectrale et de fusion, dont :

- La création de **stacks multispectraux** (empilement de bandes)
- L'affichage, le traitement et l'amélioration d'images raster
- Les **classifications supervisées et non supervisées** (Maximum Likelihood, ISODATA, Parallelepiped...)
- Le calcul d'**indices spectraux** via le Raster Calculator ou Model Maker
- L'analyse statistique des couches raster (moyenne, écart-type, histogrammes)
- Des outils de **filtrage spatial** et de **nettoyage post-classification**

II.2.2 Limites d'ERDAS Imagine 2011

Malgré sa puissance pour les traitements classiques, la version 2011 présente certaines **limites** :

- **Interface vieillissante** peu intuitive par rapport aux standards modernes
- **Absence d'outils avancés** basés sur le machine learning ou le deep learning
- **Manque de compatibilité directe** avec certains formats ou projections récents
- Faible support pour les données Sentinel-2, qui sont apparues après cette version

Pour compenser ces manques, certains traitements ont été partiellement réalisés dans un logiciel plus flexible.

II.2.3 Complémentarité avec QGIS

QGIS, logiciel open-source, a été utilisé en complément pour certaines étapes, notamment :

- La **visualisation rapide** des couches raster
- La **conversion des DN en température** pour la bande thermique Landsat
- L'extraction des valeurs statistiques (moyenne, écart-type)
- L'utilisation du **Raster Calculator** pour générer des couches binaires ou des indices
- La reprojection des images en cas d'incompatibilité

QGIS a permis de surmonter plusieurs contraintes techniques non prises en charge nativement par ERDAS Imagine 2011.

II.2.4 Méthodologie générale adoptée

L'approche suivie se base sur le traitement séparé de deux types d'images satellitaires, chacun correspondant à un objectif spécifique :

- **Sentinel-2** (résolution élevée) pour la **détection des panneaux solaires** par classification supervisée
- **Landsat 8 (B10)** pour la **détection des points chauds (hotspots)** à partir de la température de surface

Les deux traitements ont été menés indépendamment, puis doivent être croisés pour une **interprétation combinée**, malgré les différences de résolution et de projection.

II.3 Prétraitement et contraintes :

Chaque image a fait l'objet d'un prétraitement avant l'analyse. Les principales étapes ont été :

- **Correction radiométrique** : conversion des DN en réflectance (Sentinel-2 : les **valeurs sont déjà corrigées** (réflectance de surface)) ou (Landsat TIRS : donne des valeurs brutes DN, il faut effectuer une conversion depuis DN → Radiance → Température (°C)).

Pour corriger les valeurs brutes de la bande thermique (B10) de l'image **Landsat 8**, nous avons utilisé les **paramètres contenus dans le fichier MTL.txt** associé à l'image. Ce fichier contient toutes les **informations nécessaires à la conversion des valeurs numériques (DN) en température réelle** principalement : K1_CONSTANT_BAND_10, K2_CONSTANT_BAND_10, RADIANCE_MULT_BAND_10, RADIANCE_ADD_BAND_10.

```
GROUP = LEVEL1_THERMAL_CONSTANTS
K1_CONSTANT_BAND_10 = 799.0284
K2_CONSTANT_BAND_10 = 1329.2405
```

Figure 2: Extrait du fichier MTL

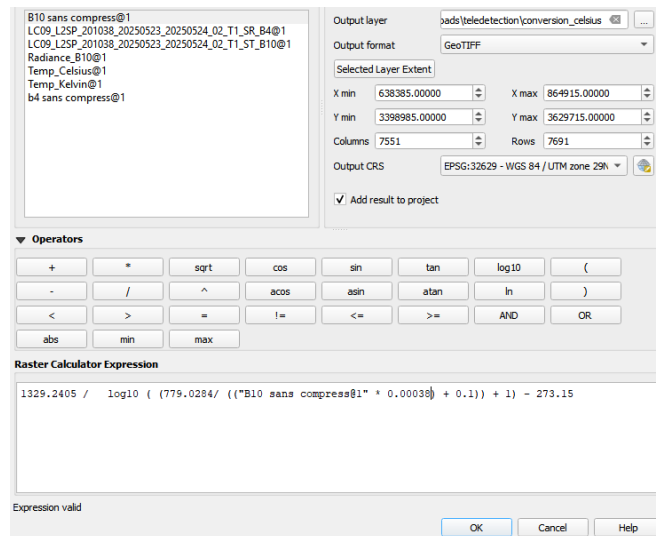


Figure 3: conversion DN/Celsius

- Correction géométrique: vérification que les deux jeux de données utilisent le même système de coordonnées et couvrent exactement la même zone géographique.

projection

- *Sélection de zone d'intérêt* : découpage (subset) des images sur la zone solaire étudiée afin de réduire la surface à analyser.
- *Composition de bandes* : pour Sentinel-2, les bandes 2,3,4,8 ont été alignées en une image multispectrale (Layer Stack). Pour Landsat, seule la bande thermique 10 a été utilisée.

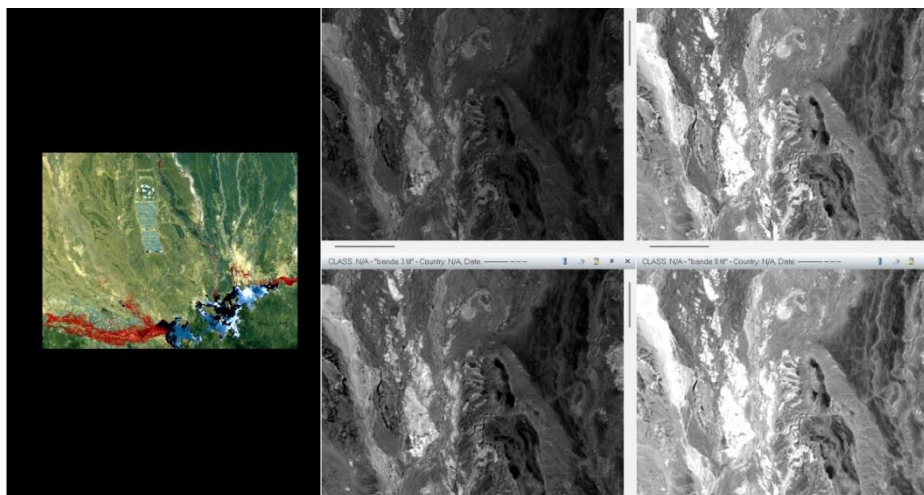


Figure 4:Resultat :layer stack

La principale contrainte fut la **disparité de résolution spatiale** entre les deux capteurs. En général, pour fusionner des images, on vise à les rendre **compatibles en résolutions** (par exemple en sur-échantillonnant la plus basse ou en sous-échantillonnant la plus haute). La littérature souligne que de nombreuses applications fusionnent les données au pas le plus fin disponible (ici 10 m). Or ici, la bande thermique Landsat est très grossière (100 m) comparée au 10 m de Sentinel-2. Cette différence

requiert un rééchantillonnage (interpolation) au préalable si l'on veut combiner les images, ce qui peut introduire de l'incertitude ou du flou.

II.4 Traitement de l'image multispectrale :

II.4.1 Application d'indices :

Nous avons procédé à une classification basée sur les indices spectraux, en particulier l'indice de végétation par différence normalisée (NDVI). Pour cela, nous avons utilisé la bande 8 (proche infrarouge) et la bande 4 (rouge) de l'image satellite, qui sont les bandes standards pour le calcul du NDVI. Une fois l'image NDVI générée et classifiée, nous avons appliqué l'outil Inquire Cursor pour mesurer la valeur NDVI au niveau d'un pixel situé à l'intérieur d'un champ de panneaux solaires. La valeur obtenue pour ce pixel est de 0,147, ce qui est relativement faible. Cette valeur confirme que la zone mesurée n'est pas végétalisée, car le NDVI d'une végétation saine dépasse généralement 0,3 à 0,6. Ainsi, cette faible valeur est cohérente avec la présence de surfaces artificielles telles que des panneaux solaires, qui ont un faible renvoi dans le proche infrarouge et n'absorbent pas de manière caractéristique comme la végétation.

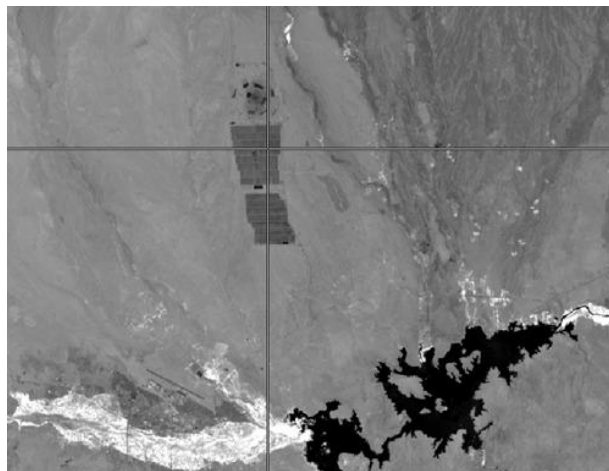


Figure 5: Résultat du NDVI

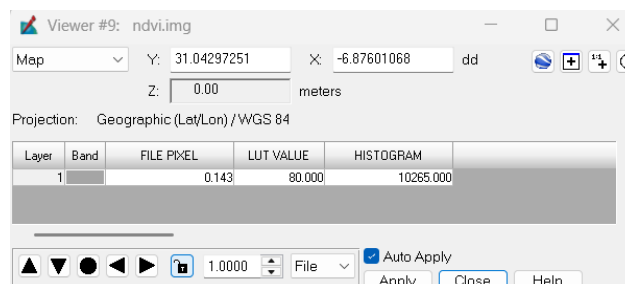
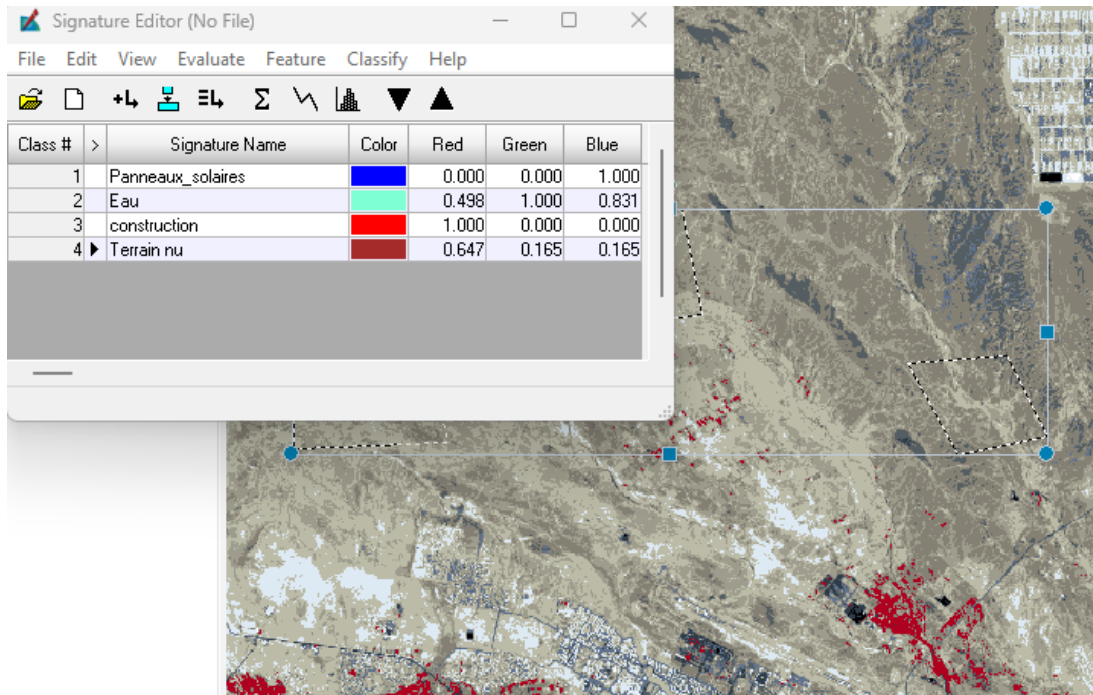


Figure 6: Valeur de l'indice au niveau d'un pixel du panneau

II.4.2 Détection des panneaux solaires (classification supervisée)

Nous avons appliqué une **classification supervisée** sur l'image Sentinel-2 pour cartographier les emprises de panneaux solaires. La procédure typique en ERDAS a été la suivante : (1) dans le

module Signature Editor (Raster → Supervised), dessiner plusieurs polygones (AOI) couvrant des panneaux solaires (identifiables à leur couleur sombre ou bleue sur le satellite) et d'autres classes du paysage (sol nu, eau, construction.) ; (2) créer une signature spectrale par classe à partir de ces AOI, puis fusionner/combinaer les signatures similaires en classes homogènes ; (3) sauvegarder le fichier .sig des signatures. Ensuite, on exécute l'outil *Supervised Classification* (Raster → Supervised Classification) en fournissant l'image Sentinel et le fichier de signatures. Le résultat est une carte thématique où chaque pixel est assigné à l'une des classes (panneau solaire, sol, veau, etc.). Comme attendu, les panneaux solaires forment une classe relativement homogène . L'avantage du superviseur est qu'il cible directement ces panneaux connus. La carte obtenue est affinée (par recodage thématique ou filtrage) pour ne retenir que la classe « panneaux ».



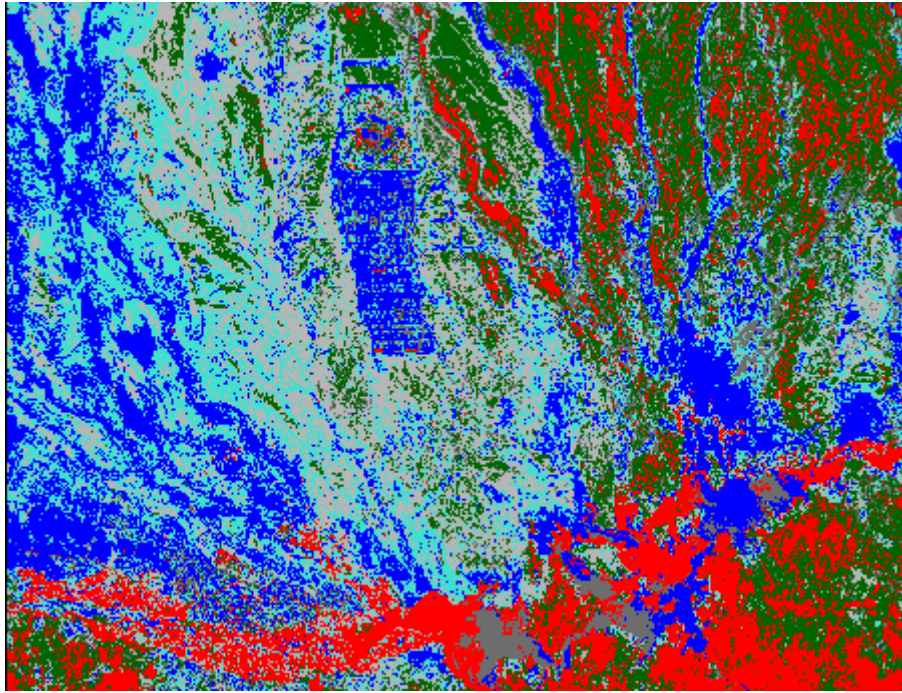


Figure 7: Resultat de classification supervisée = **mauvais résultat**

Limites rencontrées :

- La résolution (10 m) peut ne pas suffire à détecter les petits panneaux individuels.
- Les conditions d'éclairage ou l'angle de prise de vue peuvent aussi influencer la qualité de détection.

II.5 Détection des points chauds (analyse thermique)

Indépendamment, l'image thermique Landsat 8 (bande 10) a été exploitée pour repérer les points chauds. Après avoir converti la bande 10 en température de brillance (proportionnelle à la température de surface), on peut appliquer un simple **seuillage** ou une classification binaire pour isoler les pixels anormalement chauds. Par exemple, on peut déclarer « point chaud » tout pixel dont la température excède une certaine valeur seuil (moyenne + plusieurs écarts-types du site). Alternativement, une *classification non supervisée* (deux clusters : chaud vs normal) aurait pu être utilisée, les « hot pixels » formant naturellement un petit cluster aux valeurs élevées de radiance thermique. Dans tous les cas, l'idée est de générer une couche binaire (ou carte d'anomalies) localisant les pixels chauds. Notons que la résolution de 100 m de cette image limite la finesse de détection : plusieurs petits panneaux pourraient n'apparaître que comme un grand pixel chaud.

En pratique pour notre projet : En utilisant les métadonnées du fichier MTL associé à la scène. Les DN (Digital Numbers) initiaux de l'image présentaient des valeurs comprises entre 39403 et 50727, ce qui indique que l'image était au format 16 bits non calibré.

La conversion des DN en radiance a été réalisée à l'aide de la formule fournie par l'USGS :

$$L_{\lambda} = ML \times Q_{cal} + AL$$

avec :

- $ML = 0.0003342$ (valeur de `RADIANCE_MULT_BAND_10`)
- $AL = 0.1$ (valeur de `RADIANCE_ADD_BAND_10`)
- Q_{cal} étant la valeur DN.

La température de brillance a ensuite été calculée en Kelvin à l'aide de la formule suivante :

$$T = \frac{K_2}{\ln\left(\frac{K_1}{L_{\lambda}}\right)} - 273.15$$

où :

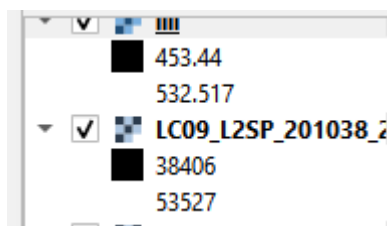
- $K_1 = 774.8853$
- $K_2 = 1321.0789$

Toutes ces constantes sont extraites directement du fichier MTL.

La formule complète utilisée dans le calculateur raster de QGIS était :

```
1321.0789 / log10((774.8853 / ((0.0003342 * "Band_10 ss compression" + 0.1)) + 1)) - 273.15
```

Malgré l'application correcte de cette méthode, les valeurs de température obtenues restent élevées, dans une plage entre 400 et 500, ce qui n'est pas cohérent avec les températures de surface attendues pour la région étudiée. Plusieurs vérifications ont été effectuées : exactitude des constantes, syntaxe dans le calculateur raster, nature des données en entrée (DN bruts), et format de sortie (`Float32`). Ce résultat suggère soit un problème d'encodage de l'image thermique initiale, soit un comportement inattendu du logiciel dans le traitement ou l'affichage des valeurs, qui devra faire l'objet d'une analyse complémentaire.



II.6 Fusion des résultats et contraintes

L'objectif final est de recouper la localisation des panneaux (Sentinel-2) avec celle des points chauds (Landsat thermique). Une option étudiée était la **fusion spatiale** des deux images. ERDAS propose des méthodes de fusion (Resolution Merge) dont la **fusion par ondelettes** (Wavelet Resolution Merge) est réputée pour préserver la radiométrie d'origine et rendre les couleurs similaires à l'image multispectrale. En théorie, on insère l'image thermique comme bande panchromatique et les bandes Sentinel comme multispectre, puis on lance le Wavelet Merge. **Cependant, cette approche a**

échoué : le module suppose que l'image « Pan » est de résolution plus fine et co-enregistrée avec les multispectres. Ici, c'est l'inverse (Sentinel 10 m est plus fin que Landsat 100 m), provoquant un désalignement non supporté directement. De plus, l'écart de résolution de 1 à 10 a empêché toute fusion automatique sans rééchantillonnage préalable. En résumé, la fusion d'images multisources a révélé que **co-registration et homogénéité spatiale** sont essentielles. Dans la pratique, on a donc traité les deux couches séparément et recoupé leurs résultats sous SIG : par exemple, on superpose la carte de panneaux et la carte de points chauds pour repérer quels panneaux coïncident avec un hotspot.

En conclusion, la démarche méthodologique a consisté à utiliser ERDAS Imagine pour exploiter séparément Sentinel-2 (classification supervisée) et Landsat 8 (analyse thermique), puis à combiner ces informations. Les étapes clés ont été : préparation et calibration des images, réalisation des classifications supervisées (détection de panneaux) et seuil (détection de chaleurs), et enfin intégration des deux jeux de résultats. Les défis principaux sont liés au prétraitement – en particulier l'alignement géométrique – et surtout à la différence de résolution entre les capteurs (10 m vs 100 m), qui nécessite de travailler sur des données homogènes ou de repenser la fusion.

III. Approche méthodologique de traitement en utilisant le deep-learning:

III.1 Définition du Deep Learning :

Le Deep Learning est un sous-domaine de l'apprentissage automatique qui utilise des réseaux de neurones artificiels profonds, c'est-à-dire composés de plusieurs couches hiérarchiques pour modéliser des représentations de données complexes à différents niveaux d'abstraction.

Il repose sur l'apprentissage de représentations hiérarchiques via des transformations non linéaires successives, ce qui permet aux modèles de capter des structures de haut niveau dans les données, telles que les objets dans des images ou les syntaxes dans le langage.

Les modèles d'apprentissage profond sont entraînés à partir de grandes quantités de données en utilisant des méthodes d'optimisation telles que la rétropropagation et des techniques comme le gradient stochastique.

III.2 U-Net :

U-Net est un modèle de deep learning conçu spécifiquement pour la segmentation d'images biomédicales, mais son architecture est très efficace pour tout type de segmentation d'image pixel par pixel.

Pourquoi choisir U-Net ?

- Architecture encoder-décoder symétrique avec des connexions de saut (skip connections) : permet de combiner informations globales et locales.

- Excellent pour les petits datasets (souvent le cas dans des contextes spécifiques comme les panneaux solaires).
- Peu de données d'entraînement nécessaires.
- Précision élevée sur des tâches de segmentation fines.

➤ **Résultat attendu :** Un masque binaire représentant les pixels où les points chauds sont présents.

III.3 Contraintes et limites de l'approche par U-Net :

III.3.1 Contraintes liées aux données d'entrée :

→ Disponibilité limitée :

En effet, il existe peu de bases de données publiques contenant des images thermiques de panneaux solaires annotées. Cela a imposé la création manuelle de masques par la suite.

→ Problème de généralisation :

Un modèle entraîné sur un petit nombre de cas peut ne pas bien fonctionner sur d'autres types de panneaux, d'environnements ou d'installations solaires ce qui a augmenté le risque de ne pas avoir de résultat pour notre projet.

III.3.2 Contraintes liées aux limites naturelles du modèle U-Net :

La contrainte la plus importante que nous avons rencontré est celle de la sensibilité à la qualité des annotations vu le fait que le modèle apprend à partir de ce qu'on lui montre. Si les annotations sont floues ou imprécises, les prédictions seront également floues. Ce qui nous a poussé à faire notre propre annotation des images.

Aussi, la dépendance à la quantité de données joue un rôle important pour le modèle U-Net ; plus le modèle est profond, plus il lui faut de données diverses et équilibrées.

III.4 Données d'entrées utilisées :

- **Images (.jpg) :**

Nous avons identifié deux datasets issus des caméras thermiques FLIR

- Dataset 1 : contient 120 images thermiques
- Dataset 2 : contient 17 images thermiques.

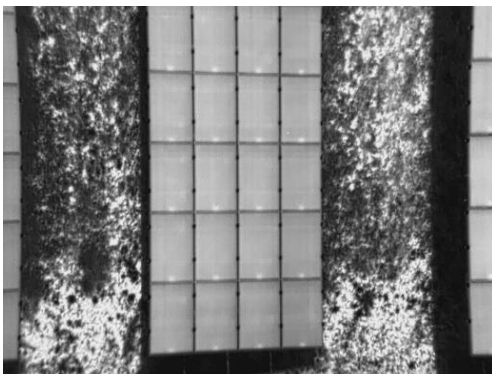


Figure 9: exemple d'image de Dataset_1

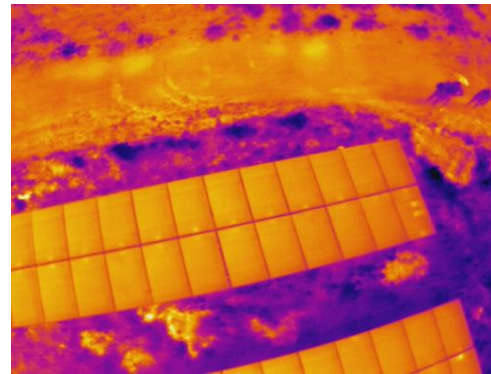


Figure 8: exemple d'image de Dataset_2

Pour résoudre la contrainte de la quantité de données et étant donné que Dataset 1 offre un nombre d'images plus élevé, nous avons décidé de travailler avec celui-ci et conserver Dataset 2 pour l'évaluation du modèle élaboré dans des parties ultérieures. Avoir plus d'images

est avantageux, surtout pour l'entraînement d'un modèle de deep learning comme U-Net, car cela permet d'améliorer la généralisation et la précision du modèle.

– **Annotations (.json) :**

Chaque fichier .json contient un objet avec une clé unique "instances" qui est une liste d'objets. Chaque objet de cette liste correspond à une zone annotée, ou "hotspot", sur une image.

Chaque instance contient 3 éléments :

- **Corners** : une liste de points {x, y} représentant les coins du polygone qui entoure la zone.
- **Center** : un point {x, y} représentant le centre géométrique de la zone.
- **defected_module** (booléen) : une indication sur l'état de la zone, true/false, signifiant zone fonctionnelle/non-défectueuse ou zone défectueuse.

```
{
  "corners": [
    {"x": 244.8372, "y": -0.1343},
    {"x": 187.0518, "y": 0.2937},
    {"x": 188.3504, "y": 93.7933},
    {"x": 246.7876, "y": 96.8234}
  ],
  "center": {
    "x": 216.7573, "y": 47.6928
  },
  "defected_module": false
}
```

III.5 Les outils et bibliothèques utiles

– **Python :**

Langage de programmation principal, très adapté au traitement d'images et au Deep Learning.

– **OpenCV / Pillow (PIL) :**

Bibliothèques pour la manipulation et le traitement d'images (lecture, écriture, redimensionnement, conversion de format, dessin).

– **Json :**

Module Python natif pour charger et manipuler les fichiers JSON contenant les annotations.

– **matplotlib.path :**

Permet de dessiner et manipuler des polygones pour convertir les annotations polygonales en masks binaires.

– **Numpy :**

Bibliothèque fondamentale pour le calcul numérique, manipulation efficace des tableaux (images, masks).

– **Pandas :**

Utile pour manipuler des données tabulaires, par exemple pour gérer des informations d'annotations, statistiques, ou suivi des résultats.

– **PyTorch ou TensorFlow/Keras :**

Frameworks de Deep Learning pour définir, entraîner, et évaluer le modèle U-Net.

– **Operating system Os :**

Outil pour manipuler les fichiers, dossiers et chemins dans Python
Il est indispensable dans tout projet qui travaille avec des images, des datasets ou des exports de résultats ce qui est indispensable pour notre projet.

III.6 Chargement des images et annotations

Préparer des paires (Image, Masque) à partir :

- Des images d'entrée
- Des annotations JSON décrivant les zones à détecter.

Le masque binaire sera une image où :

- Les zones annotées sont blanches (valeur 255),
- Le reste est noir (valeur 0)

Le masque doit avoir les mêmes dimensions (**640× 512 pixels**) que l'image source

Le masque doit contenir uniquement 0 (noir) et 255 (blanc).

Les coordonnées des coins doivent former un polygone valide et fermé.

Le masque doit être parfaitement aligné avec l'image originale :

- Pas de décalage,
- Pas de rotation ou redimensionnement mal appliqué.

Toute opération d'augmentation de données faite sur l'image doit aussi être appliquée au masque de la même manière.

III.7 Le but d'entraînement du modèle U-Net :

Le but de l'entraînement d'un modèle U-Net est de faire apprendre au modèle à segmenter correctement les images, c'est-à-dire à prédire précisément pour chaque pixel s'il appartient à la zone d'intérêt ou non. L'entraînement vise à :

- **Minimiser l'erreur entre la prédiction et la vérité terrain**

Le modèle génère des prédictions (masques) qui doivent être les plus proches possibles des masques annotés (ground truth).

- **Apprendre des caractéristiques discriminantes**

Grâce à l'encodeur et au décodeur, le modèle apprend à extraire et combiner des caractéristiques visuelles importantes (formes, textures, contours) qui lui permettent de distinguer les zones à segmenter.

- **Généraliser**

Le modèle doit être capable de segmenter correctement des images qu'il n'a jamais vues (données de test ou réelles), pas seulement celles du jeu d'entraînement.

- **Optimiser ses paramètres internes (poids)**

Par l'optimisation, l'entraînement ajuste les millions de poids du réseau pour améliorer sa capacité à faire la segmentation.

III.8 Construction et entraînement du modèle U-Net

III.8.1 Architecture U-Net :

U-Net est un réseau convolutif conçu pour la segmentation d'images, avec une architecture en forme de "U" comprenant deux parties principales :

- Encodeur (contracting path) : extrait les caractéristiques importantes de l'image en réduisant progressivement la taille spatiale à travers des couches de convolutions suivies de pooling.
- Décodeur (expansive path) : reconstruit la carte de segmentation à la taille originale en utilisant des convolutions transposées et des skip connections (raccourcis) qui permettent de récupérer les détails spatiaux perdus dans l'encodeur.

III.8.2 Initialisation :

- On a un jeu de données composé d'images (X) et de masques binaires (Y) annotés.
- Les images sont redimensionnées, normalisées, et les masques sont binaires (0 = fond, 1 = zone à segmenter).

III.8.3 Fonction de perte :

Pour entraîner efficacement le modèle U-Net, on utilise une combinaison de Binary Cross-Entropy (BCE) et Dice Loss :

- **BCE** mesure l'erreur pixel à pixel entre la prédiction et le masque réel.
- **Dice Loss** favorise une meilleure superposition des régions segmentées, ce qui est particulièrement utile lorsque les zones à détecter sont petites et peu fréquentes, comme les hotspots.

III.8.4 Optimiseur

Lorsqu'on entraîne un réseau de neurones comme U-Net, l'objectif est de minimiser la fonction de perte (qui mesure l'erreur entre la prédiction et la vérité terrain). Pour cela, on ajuste progressivement les poids internes du modèle.

L'optimiseur est l'algorithme qui décide comment modifier ces poids à chaque étape (itération) d'apprentissage, en fonction de la perte calculée.

On a Employé l'optimiseur Adam, avec un taux d'apprentissage à ajuster typiquement entre **0.001** et **0.00001**, selon les performances observées.

III.8.5 Entraînement du modèle

a. Division du jeu de données

Jeu d'entraînement (70-80 %) :

Utilisé pour entraîner le modèle. Le modèle va apprendre à partir de ces images et leurs masques associés.

Jeu de validation (20-30 %) :

Utilisé pour évaluer la performance du modèle pendant l'entraînement, sans influencer les mises à jour des poids. Cela permet de vérifier que le modèle généralise bien et ne fait pas que "mémoriser" les données d'entraînement.

b. Processus d'entraînement par époque

Une époque correspond à une passe complète sur l'ensemble des images d'entraînement.

Pour chaque époque :

Passer les images d'entraînement dans le modèle : Le modèle prédit des masques à partir des images d'entrée.

Calculer la perte :

La fonction de perte compare ces masques prédits aux masques réels (ground truth) pour quantifier l'erreur.

Rétropropagation :

À partir de la perte, le modèle calcule les gradients (dérivées) qui indiquent comment ajuster les poids pour réduire l'erreur. Ces poids sont ensuite mis à jour via l'optimiseur

Évaluer les métriques sur le jeu de validation :

Après chaque époque, on teste le modèle sur les images de validation pour calculer des métriques comme l'IoU ou le Dice score. Ces mesures permettent de suivre la qualité de la segmentation sur des données inédites.

III.8.6 Évaluation et validation

○ **Validation quantitative**

Sur le jeu de validation, calculer :

- **Intersection over Union (IoU)**
- **Dice Coefficient**

→ **IoU (Intersection over Union)**

Formule :

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

- A = masque prédictif
- B = masque réel (ground truth)
- Valeur entre 0 et 1 (1 = parfait)
- Très utilisée en **segmentation sémantique**

→ **Dice Coefficient**

Formule :

$$Dice = \frac{2|A \cap B|}{|A| + |B|}$$

- Plus tolérant aux petites erreurs que l'IoU.
- Proche de l'IoU, mais plus stable en cas de petit objet.

Dans le cas binaire : IoU + Dice sont souvent suffisants.

○ **Validation visuelle**

- Superposer les masques prédits sur les images originales pour une inspection visuelle.
- Vérifier l'absence de sur-segmentation (faux positifs) et d'omissions (faux négatifs).

III.8.7 Déploiement et utilisation

- Sauvegarder le modèle entraîné pour une utilisation future.
- Appliquer le modèle sur de nouvelles images pour prédire automatiquement les hotspots.
- Intégrer éventuellement le modèle dans une application de monitoring en temps réel des panneaux solaires pour détecter les anomalies.

IV. Segmentation des points chauds sur les panneaux solaires Par deeplearning

IV.1 Etapes :

IV.1.1 Entrainement :

L'exécution sur Visual Studio Code et on a utilisé les données de la dataset_1

```
import os
import json
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt
import numpy as np

# 1. Fonction pour convertir un masque JSON en image binaire
def json_to_mask(json_path, img_shape):
    with open(json_path, 'r') as f:
        data = json.load(f)
    mask = Image.new("L", img_shape, 0)
    if "shapes" in data:
        for shape in data["shapes"]:
            points = shape["points"]
            # Convertir les coordonnées en tuples d'entiers
            points = [tuple(map(float, pt)) for pt in points]
            ImageDraw.Draw(mask).polygon(points, outline=1, fill=1)
    return mask

# 2. Dataset adapté pour masques JSON
class HotspotDataset(Dataset):
    def __init__(self, image_dir, mask_dir, transform=None):
        self.image_dir = image_dir
        self.mask_dir = mask_dir
        self.images = [
            f for f in os.listdir(image_dir)
            if os.path.isfile(os.path.join(mask_dir, os.path.splitext(f)[0] + ".json"))
        ]
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_name = self.images[idx]
        img_path = os.path.join(self.image_dir, img_name)
        mask_name = os.path.splitext(img_name)[0] + ".json"
```

```

mask_path = os.path.join(self.mask_dir, mask_name)
image = Image.open(img_path).convert("L")
# masque à la taille de l'image
mask = json_to_mask(mask_path, image.size)
if self.transform:
    image = self.transform(image)
    mask = self.transform(mask)
mask = (mask > 0).float()
return image, mask

# 3. U-Net architecture (simplifiée)
class UNet(nn.Module):
    def __init__(self):
        super(UNet, self).__init__()
        self.enc1 = self.conv_block(1, 32)
        self.enc2 = self.conv_block(32, 64)
        self.enc3 = self.conv_block(64, 128)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = self.conv_block(128, 256)
        self.up3 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.dec3 = self.conv_block(256, 128)
        self.up2 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.dec2 = self.conv_block(128, 64)
        self.up1 = nn.ConvTranspose2d(64, 32, 2, stride=2)
        self.dec1 = self.conv_block(64, 32)
        self.final = nn.Conv2d(32, 1, 1)

    def conv_block(self, in_ch, out_ch):
        return nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        b = self.bottleneck(self.pool(e3))
        d3 = self.up3(b)
        d3 = torch.cat([d3, e3], dim=1)
        d3 = self.dec3(d3)
        d2 = self.up2(d3)
        d2 = torch.cat([d2, e2], dim=1)
        d2 = self.dec2(d2)
        d1 = self.up1(d2)
        d1 = torch.cat([d1, e1], dim=1)

```

```

        d1 = self.dec1(d1)
        out = self.final(d1)
        return torch.sigmoid(out)

# 4. Affichage d'une image, du masque et du prédit
def show_sample(image, mask, pred=None):
    image = image.squeeze().cpu().numpy()
    mask = mask.squeeze().cpu().numpy()
    plt.figure(figsize=(12,4))
    plt.subplot(1, 3, 1)
    plt.title('Image')
    plt.imshow(image, cmap='gray')
    plt.axis('off')
    plt.subplot(1, 3, 2)
    plt.title('Mask réel')
    plt.imshow(mask, cmap='gray')
    plt.axis('off')
    if pred is not None:
        pred = pred.squeeze().cpu().detach().numpy()
        plt.subplot(1, 3, 3)
        plt.title('Mask prédit')
        plt.imshow(image, cmap='gray')
        plt.imshow(pred > 0.5, alpha=0.5, cmap='jet')
        plt.axis('off')
    plt.show()

# 5. Paramètres et chemins
image_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_1\images"
mask_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_1\MASKS"
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
])

# 6. Préparation des données
dataset = HotspotDataset(image_dir, mask_dir, transform)
test_size = int(0.2 * len(dataset))
train_size = len(dataset) - test_size
train_dataset, test_dataset = torch.utils.data.random_split(dataset, [train_size, test_size])
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1, shuffle=False)

# 7. Initialisation du modèle, perte, optimiseur
model = UNet().to(device)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

```

```

# 8. Entraînement
n_epochs = 15
for epoch in range(n_epochs):
    model.train()
    running_loss = 0
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f"Epoch {epoch+1}/{n_epochs}, Loss: {running_loss/len(train_loader):.4f}")
torch.save(model.state_dict(),
r"C:\Users\PC\Desktop\2CI\projet_teledection\python\unet_hotspot.pth")
print("Modèle sauvegardé dans unet_hotspot.pth")

# 9. Sauvegarde du modèle
torch.save(model.state_dict(),
r"C:\Users\PC\Desktop\2CI\projet_teledection\python\unet_hotspot.pth")

# 10. Affichage résultats sur 5 images de test
model.eval()
with torch.no_grad():
    shown = 0
    for images, masks in test_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        show_sample(images[0], masks[0], outputs[0])
        shown += 1
        if shown == 5:
            break

# 11. Prédiction et sauvegarde des masques prédits (optionnel)
output_dir = "predicted_masks"
os.makedirs(output_dir, exist_ok=True)
model.eval()
with torch.no_grad():
    for idx, (images, masks) in enumerate(test_loader):
        images = images.to(device)
        outputs = model(images)
        pred = (outputs[0].squeeze().cpu().numpy() > 0.5).astype(np.uint8) * 255
        pil_img = Image.fromarray(pred)
        pil_img.save(os.path.join(output_dir, f"pred_{idx}.png"))

```

IV.1.2 Affichage :

Le code donne résultat sur 5 images parmi les 120 images de dataset1 , une après l'autre comme suit :

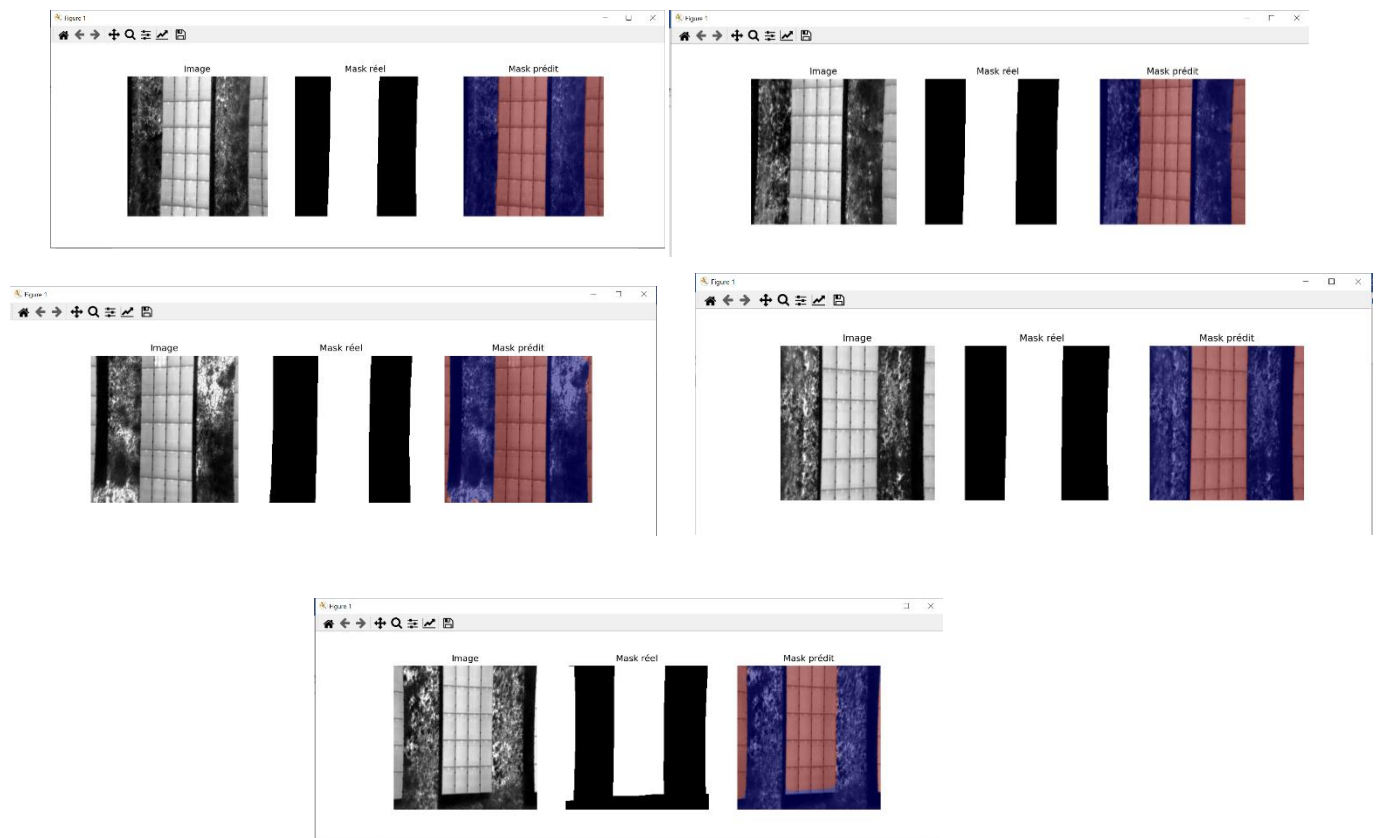


Figure 10; résultat de l'entraînement

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/PC/Desktop/2CI/projet_teledection/python/EntrainementXX.py"
Epoch 1/15, Loss: 0.6841
Epoch 2/15, Loss: 0.6613
Epoch 3/15, Loss: 0.3861
Epoch 4/15, Loss: 0.1819
Epoch 5/15, Loss: 0.1489
Epoch 6/15, Loss: 0.1204
Epoch 7/15, Loss: 0.0932
Epoch 8/15, Loss: 0.0862
Epoch 9/15, Loss: 0.0796
Epoch 10/15, Loss: 0.0815
Modèle sauvegardé dans unet_hotspot.pth
PS C:\Users\PC>
```

```
r"C:\Users\PC\Desktop\2CI\projet_teledection\python\UNET_hotspot.pth"
```

- On Utilisera le modèle entraîné (UNET_hotspot.pth) pour prédire sur des nouvelles images.
- La valeur de la fonction de perte (loss) mesure l'erreur entre :

la prédiction du modèle (ex. masque prédit) et la vérité terrain (ex. masque réel).

Plus elle est basse, plus le modèle est précis.

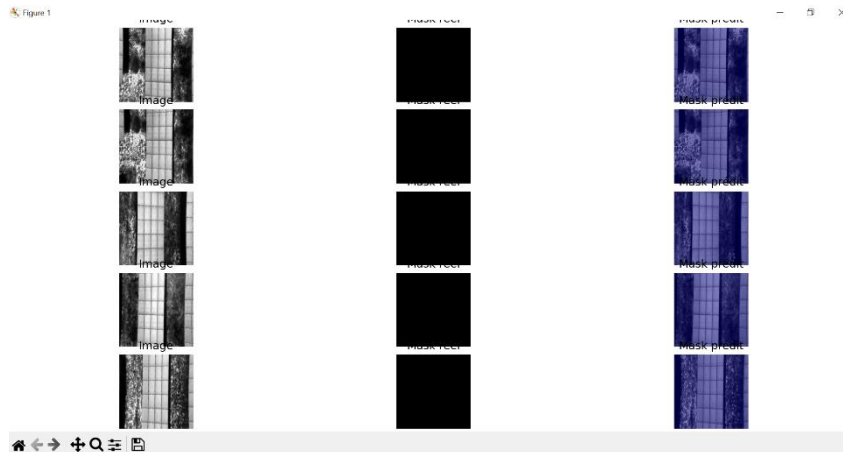
→ Une perte de 0.0408 est généralement assez faible, ce qui peut indiquer que le modèle apprend correctement.

- Notre entraînement se déroulera selon 15 epoch (15 itérations d'entraînement)

- Une epoch = un passage complet sur tout la dataset_1 d'entraînement.
- On a choisi 15 epoch car 10 à 30 epochs suffisent souvent pour des tâches simples.

IV.1.3 Problème rencontré dans la partie d'entraînement :

Au début le premier résultat qu'on obtient est :



On remarque que le masque réel (milieu) est tout noir et le masque prédit ressemble à l'image cela revient à :

- Lors la vérification automatique par python du dossier MASKS (dossier des masques réels), le script nous a confirmé que tous les fichiers .json sont vides :
- Leur clé « shapes » est une liste vide Les masques ne sont pas annotés. Donc le modèle n'apprendra jamais à détecter de “hotspots” ou d'anomalies, car il n'a AUCUN exemple de ce qu'il doit chercher.

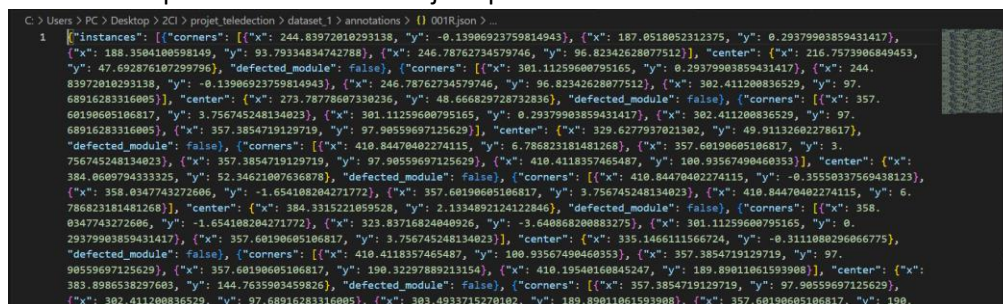
D'une autre manière, si aucun masque n'a de pixels positifs ("blancs") :

→ le modèle ne voit jamais ce que c'est un vrai hotspot

→ il apprend juste à tout prédire à zéro (fond noir).

IV.1.4 Solutions :

- Lorsqu'on ouvre un fichier .json par VS code on trouve :



- On transforme tout le dossier MASKS des fichiers .json en un format LabelMe automatiquement par le code suivant :

```
import os
import json

# === Met ici le chemin de ton dossier contenant les JSON à convertir ===
input_folder = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_1\MASKS"

# Pour chaque fichier .json du dossier
for filename in os.listdir(input_folder):
```

```

if filename.endswith(".json") and not filename.endswith("_labelme.json"):
    input_path = os.path.join(input_folder, filename)
    output_path = os.path.join(
        input_folder,
        filename.replace(".json", "_labelme.json")
    )
    try:
        with open(input_path, "r") as f:
            data = json.load(f)

        shapes = []
        for instance in data.get("instances", []):
            corners = instance.get("corners", [])
            points = [[pt["x"], pt["y"]] for pt in corners]
            if points:
                shapes.append({
                    "label": "module",
                    "points": points
                })

        labelme_data = {"shapes": shapes}

        with open(output_path, "w") as f:
            json.dump(labelme_data, f, indent=2)

        print(f"Converti : {filename} --> {os.path.basename(output_path)}")
    except Exception as e:
        print(f"Erreur pour {filename} : {e}")

print("Conversion terminée pour tout le dossier !")

```

- On obtient des fichiers .json mais renommés différemment de leurs images donc pour renommer les 120 fichiers on a utilisé ce script :

```

import os

# == À MODIFIER : dossier où sont tes images et masques ==
image_folder = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_1\IMAGES"
mask_folder = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_1\MASKS"

# Extension des images (modifie si besoin)
image_ext = ".jpg" # ou ".png" etc.

# 1. Liste les images du dossier
image_names = [f for f in os.listdir(image_folder) if f.lower().endswith(image_ext)]

# 2. Pour chaque image, cherche son masque et renomme-le
for img_name in image_names:
    base = os.path.splitext(img_name)[0] # ex: '001R'
    old_mask = os.path.join(mask_folder, f"{base}_labelme.json")

```

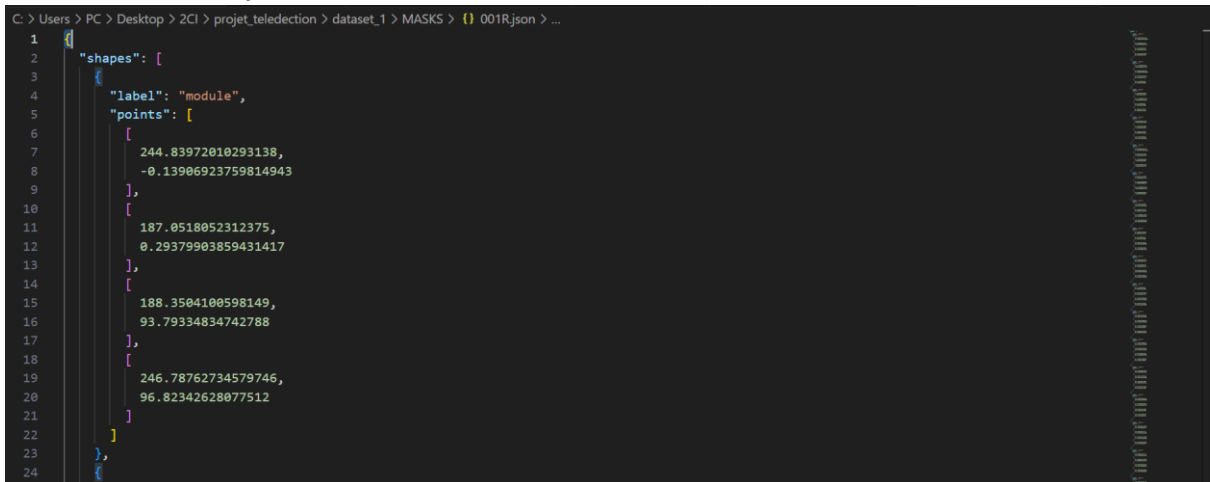
```

new_mask = os.path.join(mask_folder, f"{base}.json")
if os.path.exists(old_mask):
    os.rename(old_mask, new_mask)
    print(f"{old_mask} -> {new_mask}")
else:
    print(f"Pas trouvé : {old_mask}")

print("Renommage terminé !")

```

- Les fichiers .json sont finalement annotés, ils sont conforme au format LabelMe :



```

1  {
2    "shapes": [
3      {
4        "label": "module",
5        "points": [
6          [
7            244.83972010293138,
8            -0.13906923759814943
9          ],
10         [
11           187.0518052312375,
12           0.29379903859431417
13         ],
14         [
15           188.3504100598149,
16           93.79334834742788
17         ],
18         [
19           246.78762734579746,
20           96.82342628077512
21         ]
22       ]
23     ],
24   }

```

IV.1.5 Evaluation du modèle :

Tester la performance du modèle par prédictions sur de nouvelles images “phase de test sur images inconnues”.

L'évaluation est faite sur des nouvelles images de dataset_2 et non pas les images d'entraînement de dataset_1; on a 17 images avec 17 masques .json.

Ce script :

1-Fait de la prédiction

2-Visualise les résultats

3-Évalue la qualité du modèle

-Pour chaque image de test :

- Lecture du masque réel (json → image binaire)
- Lecture du masque prédit (png → image binaire)
- Calcul de IoU et Dice

-Affichage des scores pour chaque image

-Affichage de la moyenne globale

4-Sauvegarde les métriques dans un fichier Excel et un fichier CSV.

on a choisie de travailler avec les métriques IoU et DICE sont deux mesures très courantes pour évaluer la qualité de la segmentation d'images.

	IoU	Dice
Formule	Intersection / Union	$2 \times \text{Intersection} / (A + B)$
Valeur max	1 (parfait)	1 (parfait)
Sensibilité	Plus sévère/stricte	Plus indulgent, plus « souple »
Interprétation	% de recouvrement exact	Mesure la similarité globale

- IoU et Dice comparent le recouvrement entre deux masques.
- IoU : plus strict, % de recouvrement exact.
- Dice : plus tolérant, mesure de similarité.
- Les deux sont entre 0 et 1 (plus c'est proche de 1, mieux c'est).

```
import os
import torch
from torchvision import transforms
from PIL import Image, ImageDraw
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import json

# == 1. Définition du modèle UNet ==
import torch.nn as nn

class UNet(nn.Module):
    def __init__(self):
        super(UNet, self).__init__()
        self.enc1 = self.conv_block(1, 32)
        self.enc2 = self.conv_block(32, 64)
        self.enc3 = self.conv_block(64, 128)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = self.conv_block(128, 256)
        self.up3 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.dec3 = self.conv_block(256, 128)
        self.up2 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.dec2 = self.conv_block(128, 64)
        self.up1 = nn.ConvTranspose2d(64, 32, 2, stride=2)
        self.dec1 = self.conv_block(64, 32)
        self.final = nn.Conv2d(32, 1, 1)

    def conv_block(self, in_ch, out_ch):
        return nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.ReLU(inplace=True),
```

```

        nn.Conv2d(out_ch, out_ch, 3, padding=1),
        nn.ReLU(inplace=True),
    )

def forward(self, x):
    e1 = self.enc1(x)
    e2 = self.enc2(self.pool(e1))
    e3 = self.enc3(self.pool(e2))
    b = self.bottleneck(self.pool(e3))
    d3 = self.up3(b)
    d3 = torch.cat([d3, e3], dim=1)
    d3 = self.dec3(d3)
    d2 = self.up2(d3)
    d2 = torch.cat([d2, e2], dim=1)
    d2 = self.dec2(d2)
    d1 = self.up1(d2)
    d1 = torch.cat([d1, e1], dim=1)
    d1 = self.dec1(d1)
    out = self.final(d1)
    return torch.sigmoid(out)

# === 2. Fonctions utilitaires ===

def json_to_mask(json_path, img_shape):
    with open(json_path, 'r') as f:
        data = json.load(f)
    mask = Image.new("L", img_shape, 0)
    if "shapes" in data:
        for shape in data["shapes"]:
            points = shape["points"]
            points = [tuple(map(float, pt)) for pt in points]
            ImageDraw.Draw(mask).polygon(points, outline=1, fill=1)
    return np.array(mask)

def iou_score(y_true, y_pred):
    intersection = np.logical_and(y_true, y_pred).sum()
    union = np.logical_or(y_true, y_pred).sum()
    return intersection / union if union != 0 else 1.0

def dice_score(y_true, y_pred):
    intersection = np.logical_and(y_true, y_pred).sum()
    total = y_true.sum() + y_pred.sum()
    return 2 * intersection / total if total != 0 else 1.0

# === 3. Chemins à adapter ===
model_path = r"C:\Users\PC\Desktop\2CI\projet_teledection\python\unet_hotspot.pth"
input_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_2\images" # Images
à prédire

```

```

output_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\masques_predits_nouvelles"
os.makedirs(output_dir, exist_ok=True)

# Pour l'évaluation (optionnel)
mask_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_2\MASKS"      #
Masques réels si disponibles

# === 4. Chargement du modèle ===
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = UNet().to(device)
model.load_state_dict(torch.load(model_path, map_location=device))
model.eval()

# === 5. Transformation des images ===
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
])

# === 6. Prédiction et sauvegarde des masques ===
image_names = [f for f in os.listdir(input_dir) if f.lower().endswith(('.png',
'.jpg', '.jpeg', '.tif'))]
for img_name in image_names:
    img_path = os.path.join(input_dir, img_name)
    image = Image.open(img_path).convert("L")
    input_tensor = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        output = model(input_tensor)
        mask_pred = (output[0].squeeze().cpu().numpy() > 0.5).astype(np.uint8) * 255
        mask_img = Image.fromarray(mask_pred)
        mask_img.save(os.path.join(output_dir, f"mask_pred_{img_name}"))

print(f"\n{len(image_names)} masques prédicts sauvegardés dans : {output_dir}")

# === 7. Visualisation (optionnelle) ===
afficher = True # Passe à False si on ne veut pas afficher
if afficher:
    for img_name in image_names[:5]: # Affiche les 5 premiers
        img_path = os.path.join(input_dir, img_name)
        mask_path = os.path.join(output_dir, f"mask_pred_{img_name}")
        image = Image.open(img_path).convert("L")
        mask_pred = Image.open(mask_path)
        plt.figure(figsize=(8,4))
        plt.subplot(1,2,1)
        plt.title('Image')
        plt.imshow(image, cmap='gray')
        plt.axis('off')
        plt.subplot(1,2,2)

```

```

plt.title('Masque prédit')
plt.imshow(image, cmap='gray')
plt.imshow(mask_pred, alpha=0.5, cmap='jet')
plt.axis('off')
plt.show()

# == 8. Évaluation quantitative si masques réels disponibles ==
# On suppose que les masques réels ont le même nom que les images avec .json
evaluer = os.path.exists(mask_dir) and len(os.listdir(mask_dir)) > 0
if evaluer:
    ious = []
    dices = []
    rows = []
    for img_name in image_names:
        base_name = os.path.splitext(img_name)[0]
        json_name = base_name + ".json"
        json_path = os.path.join(mask_dir, json_name)
        pred_path = os.path.join(output_dir, f"mask_pred_{img_name}")

        if not os.path.exists(json_path):
            print(f"Pas de masque réel pour {img_name}")
            continue

        # Masque réel (converti en image binaire)
        mask = json_to_mask(json_path, (256, 256))
        mask = (mask > 0).astype(np.uint8)

        # Masque prédit
        pred = np.array(Image.open(pred_path).resize((256, 256)))
        if pred.ndim == 3:
            pred = pred[:, :, 0]
        pred = (pred > 0).astype(np.uint8)

        iou = iou_score(mask, pred)
        dice = dice_score(mask, pred)
        ious.append(iou)
        dices.append(dice)
        rows.append({
            "Image": img_name,
            "IoU": iou,
            "Dice": dice
        })
    print(f"{img_name}: IoU={iou:.3f}, Dice={dice:.3f}")

# Résumé
if ious:
    iou_moy, iou_std = np.mean(ious), np.std(ious)
    dice_moy, dice_std = np.mean(dices), np.std(dices)

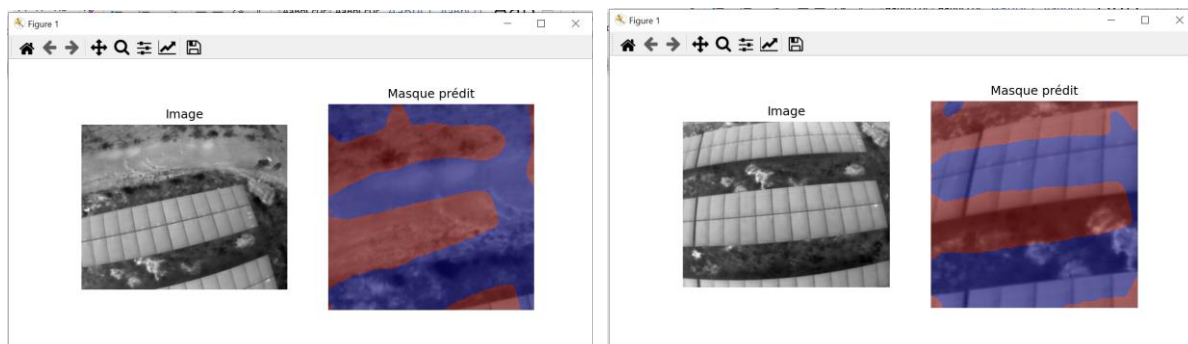
```

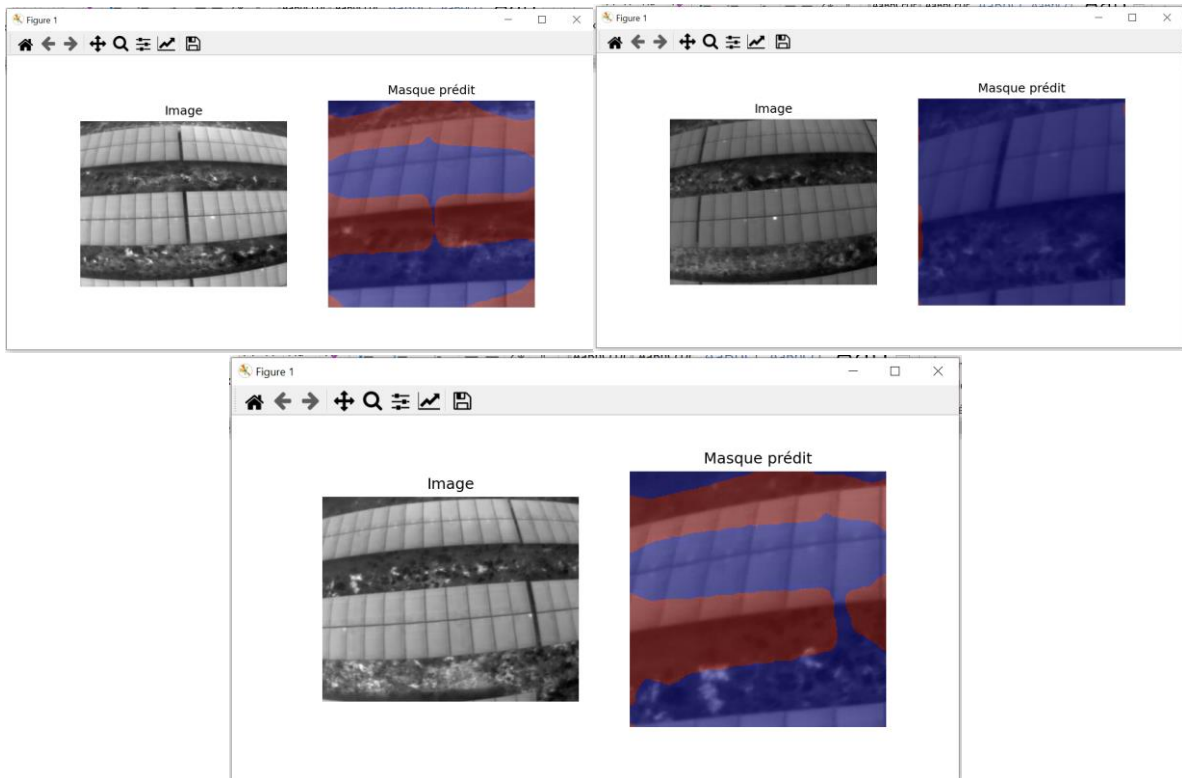
```

print(f"\nIoU moyen : {iou_moy:.3f} ( $\pm$ {iou_std:.3f})")
print(f"Dice moyen : {dice_moy:.3f} ( $\pm$ {dice_std:.3f})")
# Sauvegarde des résultats
rows.append({
    "Image": "MOYENNE",
    "IoU": iou_moy,
    "Dice": dice_moy
})
rows.append({
    "Image": "ECART-TYPE",
    "IoU": iou_std,
    "Dice": dice_std
})
df = pd.DataFrame(rows)
df.to_csv(os.path.join(output_dir, "resultats_comparaison.csv"),
index=False)
df.to_excel(os.path.join(output_dir, "resultats_comparaison.xlsx"),
index=False)
print(f"Résultats sauvegardés dans {output_dir}/resultats_comparaison.csv et
.xlsx")
else:
    print("Aucune métrique calculée (pas de masques réels correspondants).")
else:
    print("Aucun masque réel trouvé, évaluation quantitative sautée.")

```

IV.1.6 Visualisation :





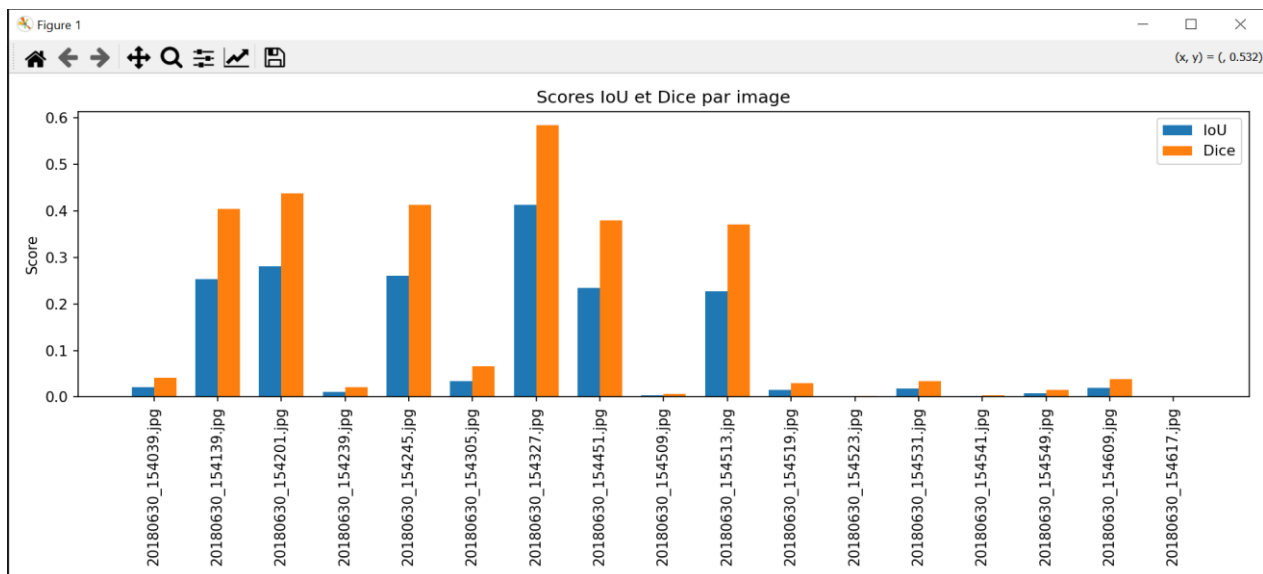
```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/PC/Desktop/2CI/projet_teledection/python/apresentrenement.py"

17 masques pr dits sauvegard s dans : C:\Users\PC\Desktop\2CI\projet_teledection\masques_predits_nouvelles
20180630_154039.jpg: IoU=0.021, Dice=0.042
20180630_154139.jpg: IoU=0.253, Dice=0.404
20180630_154201.jpg: IoU=0.281, Dice=0.438
20180630_154239.jpg: IoU=0.011, Dice=0.021
20180630_154245.jpg: IoU=0.260, Dice=0.413
20180630_154305.jpg: IoU=0.034, Dice=0.065
20180630_154327.jpg: IoU=0.413, Dice=0.584
20180630_154451.jpg: IoU=0.234, Dice=0.380
20180630_154509.jpg: IoU=0.003, Dice=0.007
20180630_154513.jpg: IoU=0.228, Dice=0.371
20180630_154519.jpg: IoU=0.015, Dice=0.030
20180630_154523.jpg: IoU=0.001, Dice=0.002
20180630_154531.jpg: IoU=0.017, Dice=0.034
20180630_154541.jpg: IoU=0.002, Dice=0.003
20180630_154549.jpg: IoU=0.008, Dice=0.015
20180630_154609.jpg: IoU=0.019, Dice=0.038
20180630_154617.jpg: IoU=0.000, Dice=0.000

IoU moyen : 0.106 ( 0.133)
Dice moyen : 0.168 ( 0.200)
R sultats sauvegard s dans C:\Users\PC\Desktop\2CI\projet_teledection\masques_predits_nouvelles/resultats_comparaison.csv et .xlsx
PS C:\Users\PC>
```

IV.1.7 Analyse quantitative :

Image	IoU	Dice
20180630_154039.jpg	0.021259894	0.04163464
20180630_154139.jpg	0.253262713	0.4041654
20180630_154201.jpg	0.280718954	0.438377137
20180630_154239.jpg	0.01070255	0.021178438
20180630_154245.jpg	0.259978168	0.412670909
20180630_154305.jpg	0.033849357	0.065482184
20180630_154327.jpg	0.412904483	0.584476145
20180630_154451.jpg	0.23427409	0.379614369
20180630_154509.jpg	0.003413054	0.00680289
20180630_154513.jpg	0.227949487	0.371268509
20180630_154519.jpg	0.015116897	0.029783559
20180630_154523.jpg	0.001147249	0.00229187
20180630_154531.jpg	0.017262785	0.033939676
20180630_154541.jpg	0.001739274	0.003472509
20180630_154549.jpg	0.00763629	0.015156838
20180630_154609.jpg	0.019401878	0.03806522
20180630_154617.jpg	0	0
MOYENNE	0.105918654	0.167551782
ECART-TYPE	0.132806767	0.200349861



- IoU moyen de 0.166 signifie qu'en moyenne, seulement ~16.6 % des pixels prédits comme « point chaud » correspondent au vrai masque.
- Dice moyen de 0.168 indique un recouvrement faible (similaire à l'IoU).
- Ces scores sont assez faibles, ce qui signifie que le modèle a peu détecté correctement les zones de points chauds dans les nouvelles images.
- La grande variance (± 0.200 pour Dice) montre que certaines images sont mieux segmentées que d'autres, mais la qualité reste globalement insuffisante.
- L'image 20180630_154327.jpg est la mieux segmentée, avec les scores les plus élevés (IoU ≈ 0.41 , Dice ≈ 0.58).
- Plusieurs images ont des scores quasiment nuls (ex : 20180630_154523.jpg, 20180630_154541.jpg, 20180630_154617.jpg), ce qui signifie que le modèle n'a pratiquement rien détecté de pertinent sur ces images.
- Les valeurs du Dice sont systématiquement supérieures à celles de l'IoU, ce qui est normal car la métrique Dice est plus "tolérante" (plus facile à augmenter).

Image	Commentaire
20180630_154039.jpg	Mauvaise détection : la prédiction ne correspond presque pas au masque réel, très peu de bonne segmentation.
20180630_154139.jpg	Résultat correct : le modèle a détecté une partie significative des zones annotées, mais des erreurs subsistent.
20180630_154201.jpg	Bon résultat : la prédiction recouvre une bonne portion du masque réel, la segmentation est globalement correcte.
20180630_154239.jpg	Très faible : prédiction quasi-inexistante ou hors de la zone d'intérêt, échec de segmentation.
20180630_154245.jpg	Correct : bonne détection des zones principales, manque de précision sur certains détails.
20180630_154305.jpg	Faible : la prédiction couvre très peu le masque réel, la plupart des zones sont manquées.
20180630_154327.jpg	Très bon résultat : forte superposition entre prédiction et annotations, très bonne segmentation.
20180630_154451.jpg	Moyen : le modèle détecte partiellement les bonnes zones, mais avec des oublis ou du bruit.
20180630_154509.jpg	Nul : aucune correspondance, la prédiction est vide ou totalement erronée.
20180630_154513.jpg	Moyen : tendance correcte mais la segmentation reste imprécise.
20180630_154519.jpg	Très faible : peu ou pas de bonne correspondance avec l'annotation réelle.
20180630_154523.jpg	Nul : le modèle n'a rien détecté de pertinent.
20180630_154531.jpg	Très faible : la prédiction ne couvre presque rien du masque réel.
20180630_154541.jpg	Nul : aucune zone correctement détectée.
20180630_154549.jpg	Très faible : segmentation quasi absente ou hors sujet.
20180630_154609.jpg	Très faible : peu ou pas de recouvrement avec le masque réel.
20180630_154617.jpg	Nul : aucune prédiction correcte, échec total sur cette image.

IV.1.8 Comparaison entre le masque réel et le masque predict:

IV.1.8.1 Code :

```
import os
import json
import numpy as np
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt

def json_to_mask(json_path, img_shape):
    with open(json_path, 'r') as f:
        data = json.load(f)
    mask = Image.new("L", img_shape, 0)
    if "shapes" in data:
        for shape in data["shapes"]:
            points = shape["points"]
            points = [tuple(map(float, pt)) for pt in points]
            ImageDraw.Draw(mask).polygon(points, outline=1, fill=1)
    return np.array(mask)

def iou_score(y_true, y_pred):
    intersection = np.logical_and(y_true, y_pred).sum()
    union = np.logical_or(y_true, y_pred).sum()
    return intersection / union if union != 0 else 1.0

def dice_score(y_true, y_pred):
    intersection = np.logical_and(y_true, y_pred).sum()
    total = y_true.sum() + y_pred.sum()
    return 2 * intersection / total if total != 0 else 1.0

# === Chemins à adapter ===
input_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_2\images"
mask_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\dataset_2\MASKS"
pred_dir = r"C:\Users\PC\Desktop\2CI\projet_teledection\masques_predictions_nouvelles"

img_shape = (256, 256) # à adapter si besoin

image_names = [f for f in os.listdir(input_dir) if f.lower().endswith(('.png',
'.jpg', '.jpeg', '.tif'))]

for img_name in image_names:
    base_name = os.path.splitext(img_name)[0]
    img_path = os.path.join(input_dir, img_name)
    json_path = os.path.join(mask_dir, base_name + '.json')
    pred_path = os.path.join(pred_dir, f"mask_pred_{img_name}")

    if not os.path.exists(json_path):
        print(f"Pas de masque réel pour {img_name}")
```

```

        continue
    if not os.path.exists(pred_path):
        print(f"Pas de masque prédit pour {img_name}")
        continue

    # Charger image, masque réel, masque prédit
    image = Image.open(img_path).convert("L").resize(img_shape)
    mask_real = json_to_mask(json_path, img_shape)
    mask_real = (mask_real > 0).astype(np.uint8)
    mask_pred = np.array(Image.open(pred_path).resize(img_shape))
    if mask_pred.ndim == 3:
        mask_pred = mask_pred[:, :, 0]
    mask_pred = (mask_pred > 0).astype(np.uint8)

    # Afficher les sommes (pixels positifs)
    print(f"\n--- {img_name} ---")
    print(f"Pixels positifs (réel): {mask_real.sum()} | (prédit): {mask_pred.sum()}")

    # Calculer IoU/Dice
    iou = iou_score(mask_real, mask_pred)
    dice = dice_score(mask_real, mask_pred)
    print(f"IoU: {iou:.4f} | Dice: {dice:.4f}")

    # Affichage visuel
    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1)
    plt.title("Image")
    plt.imshow(image, cmap="gray")
    plt.axis("off")

    plt.subplot(1,3,2)
    plt.title("Masque réel")
    plt.imshow(mask_real, cmap="gray")
    plt.axis("off")

    plt.subplot(1,3,3)
    plt.title("Masque prédit")
    plt.imshow(mask_pred, cmap="gray")
    plt.axis("off")

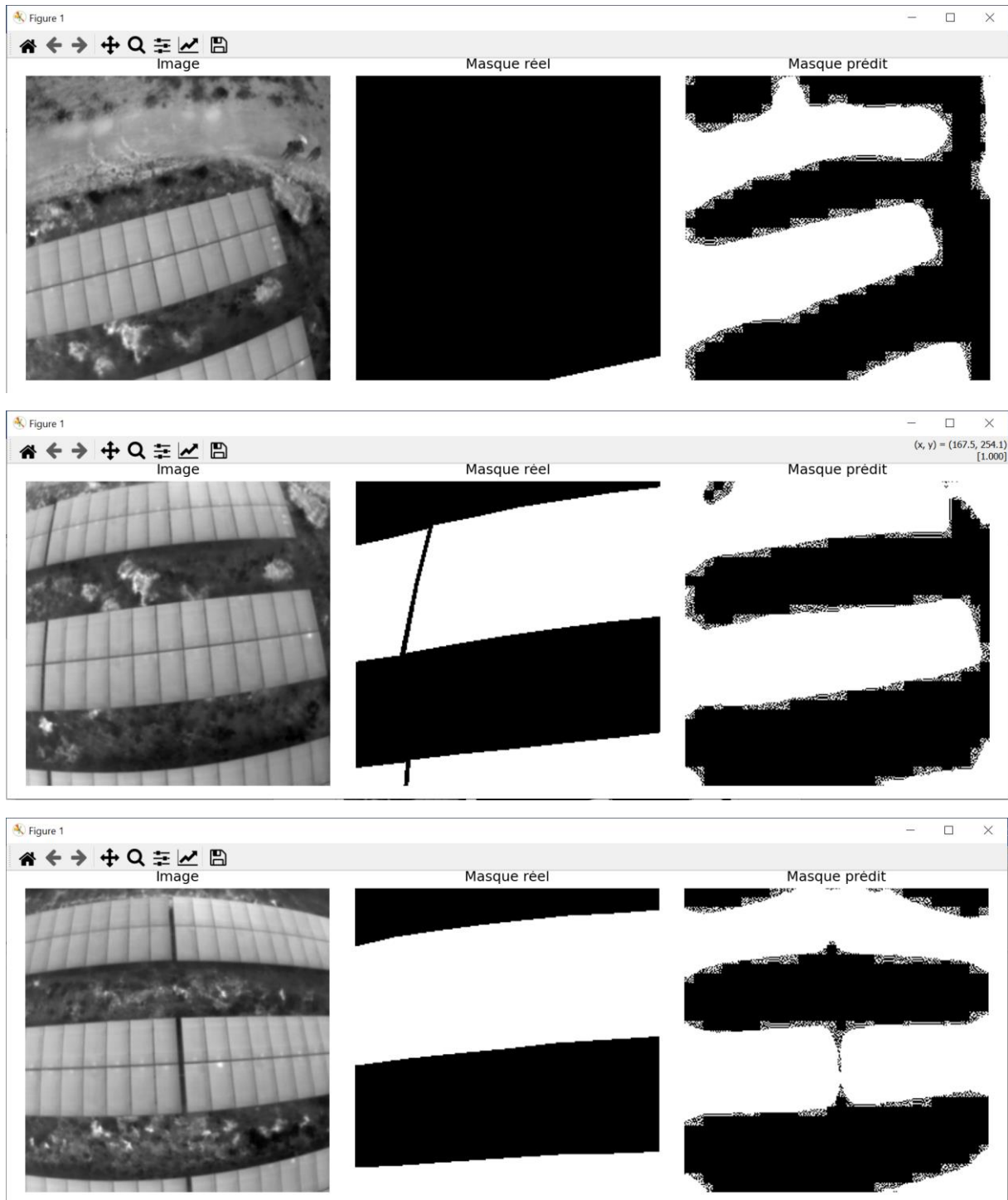
    plt.tight_layout()
    plt.show()

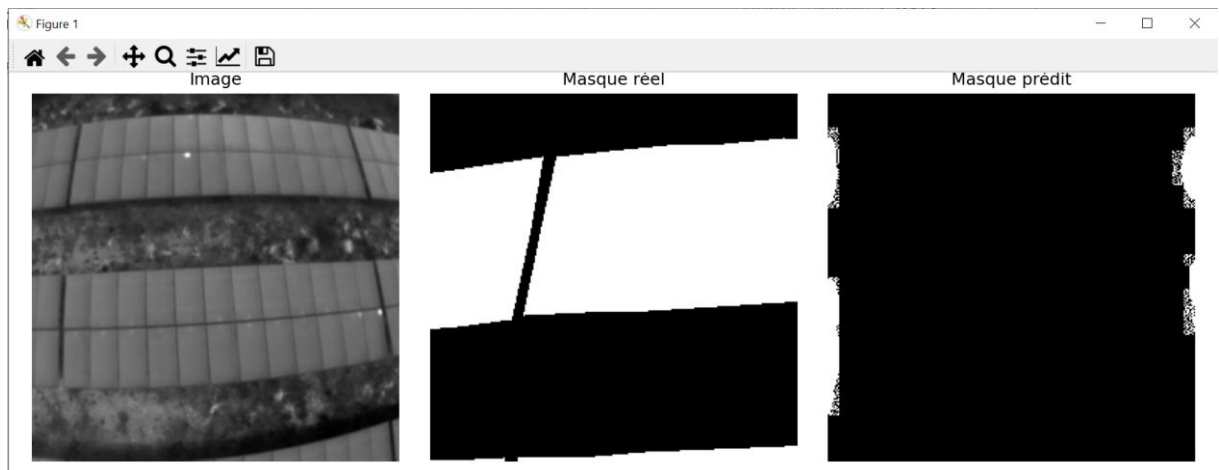
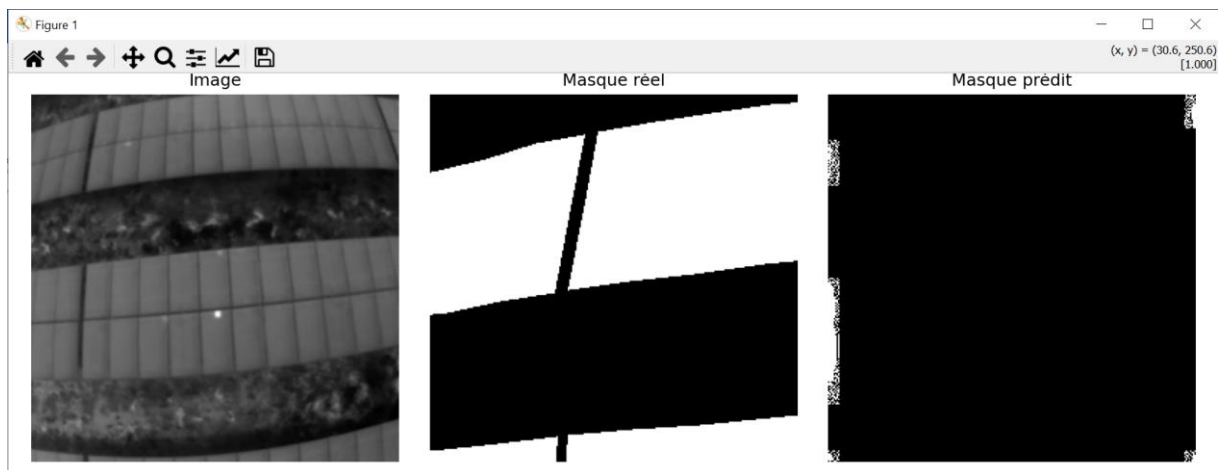
    # Pour ne traiter qu'un exemple à la fois, dé-commente la ligne suivante :
    # break

```

IV.1.8.2 Affichage :

Le script génère un résultat pour toutes les 17 images (on a pris que 6 images pour montrer le résultats) :





```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/PC/Desktop/2CI/projet teledetection/python/problemedes0ex1.py"

--- 20180630_154039.jpg ---
Pixels positifs (réel): 975 | (prédit): 30249
IoU: 0.0213 | Dice: 0.0416

--- 20180630_154139.jpg ---
Pixels positifs (réel): 33996 | (prédit): 31784
IoU: 0.2533 | Dice: 0.4042

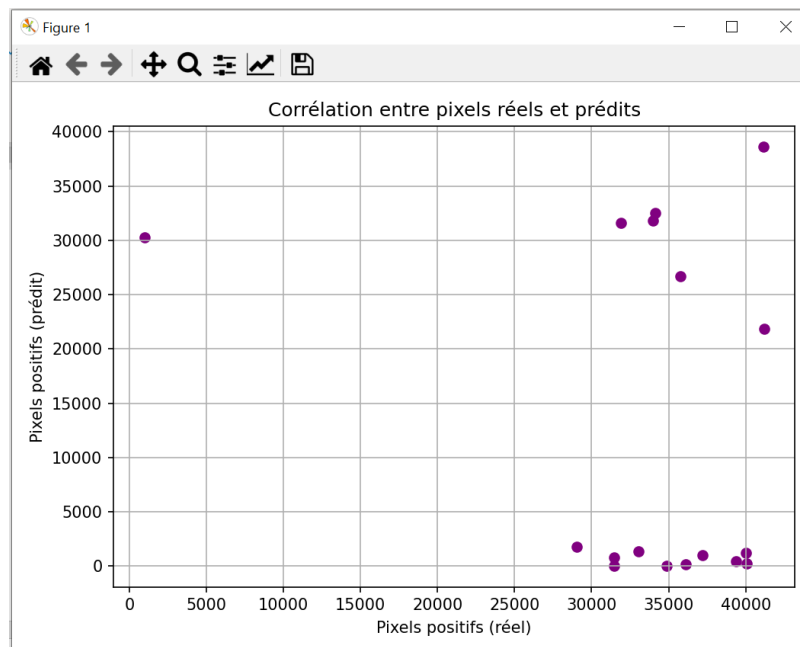
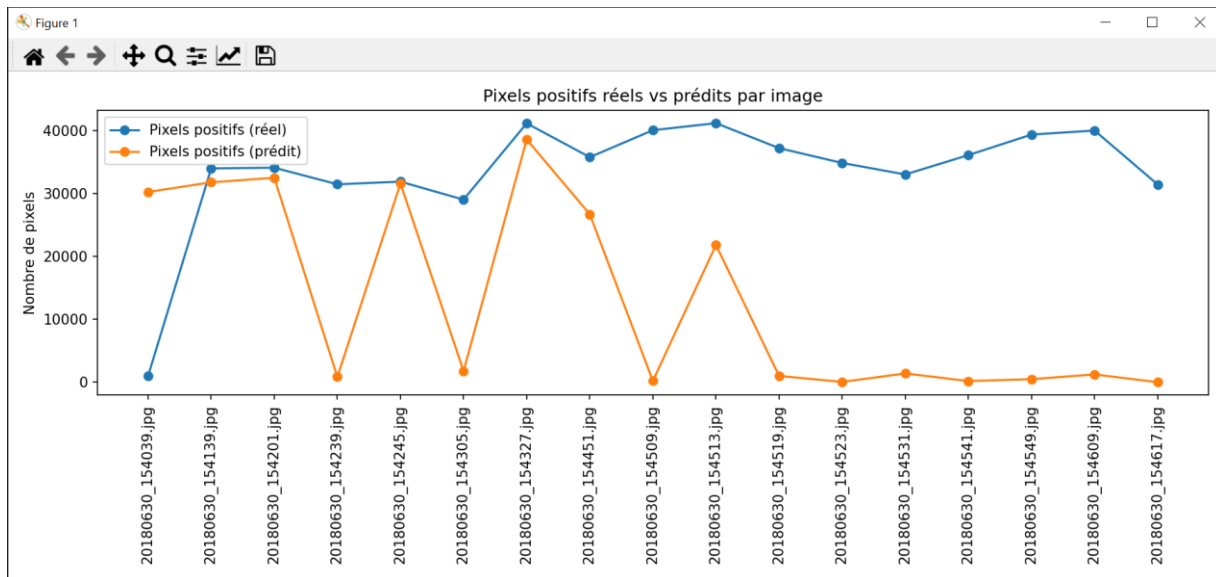
--- 20180630_154201.jpg ---
Pixels positifs (réel): 34114 | (prédit): 32509
IoU: 0.2807 | Dice: 0.4384

--- 20180630_154239.jpg ---
Pixels positifs (réel): 31468 | (prédit): 829
IoU: 0.0107 | Dice: 0.0212
```

Output enregistré sur un tableau Excel :

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/PC/Desktop/2CI/projet teledection/python/excelcomp.py"
Image Pixels positifs (réel) Pixels positifs (prédit) IoU Dice
20180630_154039.jpg 975 30249 0.0213 0.0416
20180630_154139.jpg 33996 31784 0.2533 0.4042
20180630_154201.jpg 34114 32509 0.2807 0.4384
20180630_154239.jpg 31468 829 0.0107 0.0212
20180630_154245.jpg 31892 31592 0.2600 0.4127
20180630_154305.jpg 29030 1757 0.0338 0.0655
20180630_154327.jpg 41171 38603 0.4129 0.5845
20180630_154451.jpg 35786 26709 0.2343 0.3796
20180630_154509.jpg 40076 201 0.0034 0.0068
20180630_154513.jpg 41196 21815 0.2279 0.3713
20180630_154519.jpg 37220 989 0.0151 0.0298
20180630_154523.jpg 34866 40 0.0011 0.0023
20180630_154531.jpg 33034 1380 0.0173 0.0339
20180630_154541.jpg 36115 170 0.0017 0.0035
20180630_154549.jpg 39378 472 0.0076 0.0152
20180630_154609.jpg 40022 1223 0.0194 0.0381
20180630_154617.jpg 31440 0 0.0000 0.0000
PS C:\Users\PC>
```

Image	Pixels positifs (réel)	Pixels positifs (prédit)	IoU	Dice
20180630_154039.jpg	975	30249	0.0213	0.0416
20180630_154139.jpg	33996	31784	0.2533	0.4042
20180630_154201.jpg	34114	32509	0.2807	0.4384
20180630_154239.jpg	31468	829	0.0107	0.0212
20180630_154245.jpg	31892	31592	0.26	0.4127
20180630_154305.jpg	29030	1757	0.0338	0.0655
20180630_154327.jpg	41171	38603	0.4129	0.5845
20180630_154451.jpg	35786	26709	0.2343	0.3796
20180630_154509.jpg	40076	201	0.0034	0.0068
20180630_154513.jpg	41196	21815	0.2279	0.3713
20180630_154519.jpg	37220	989	0.0151	0.0298
20180630_154523.jpg	34866	40	0.0011	0.0023
20180630_154531.jpg	33034	1380	0.0173	0.0339
20180630_154541.jpg	36115	170	0.0017	0.0035
20180630_154549.jpg	39378	472	0.0076	0.0152
20180630_154609.jpg	40022	1223	0.0194	0.0381
20180630_154617.jpg	31440	0	0	0



IV.1.8.3 Interprétation :

- Beaucoup de points sont assez dispersés.
- Plusieurs points avec un nombre élevé de pixels réels (~30 000 à 40 000) ont un faible nombre de pixels prédits (près de 0 ou très bas).
- Quelques points ont des pixels prédits très élevés alors que les pixels réels sont faibles, ou inversement.
- On observe aussi des points où les pixels prédits sont très proches des pixels réels, mais ce n'est pas la majorité.
- Les clusters en bas à droite indiquent que, pour certaines images, le modèle prédit beaucoup moins de pixels que la réalité (sous-segmentation).
- Un point en haut à gauche suggère une sur-segmentation sur au moins un exemple (beaucoup de pixels prédits sans pixels réels correspondants).

- Les points ne sont pas alignés sur la diagonale, ce qui suggère que le modèle n'a pas une très bonne capacité à estimer la quantité réelle de pixels positifs.
- La dispersion importante reflète un manque de stabilité ou de régularité dans les prédictions.
- Corrélation attendue : Si le modèle prédit exactement comme la réalité, les points devraient être alignés sur la diagonale ($Y=X$).

Image	Pixels positifs (réel)	Pixels positifs (prédit)	Commentaire
20180630_154039.jpg	975	30249	Sur-segmentation : beaucoup trop de pixels prédits par rapport au réel, prédiction peu pertinente.
20180630_154139.jpg	33996	31784	Bon équilibre : le nombre de pixels prédits est proche du réel, la segmentation est réussie.
20180630_154201.jpg	34114	32509	Bon équilibre : prédiction proche du masque réel, bonne qualité globale.
20180630_154239.jpg	31468	829	Fort sous-segmentation : le modèle oublie la quasi-totalité des zones annotées.
20180630_154245.jpg	31892	31592	Très bon équilibre : résultat très satisfaisant, prédiction conforme au réel.
20180630_154305.jpg	29030	1757	Sous-segmentation : très peu de zones détectées, la plupart sont manquées.
20180630_154327.jpg	41171	38603	Excellent équilibre : prédiction très proche du réel, très bonne correspondance.
20180630_154451.jpg	35786	26709	Prédiction correcte : nombre de pixels raisonnable, mais quelques zones manquées.
20180630_154509.jpg	40076	201	Sous-segmentation extrême : le masque prédit est presque vide alors que le réel est important.
20180630_154513.jpg	41196	21815	Prédiction correcte : bonne tendance mais il manque de nombreux pixels.
20180630_154519.jpg	37220	989	Fort sous-segmentation : très peu de pixels détectés par rapport au réel.
20180630_154523.jpg	34866	40	Sous-segmentation extrême : prédiction pratiquement vide.
20180630_154531.jpg	33034	1380	Sous-segmentation : le modèle détecte très peu de zones.
20180630_154541.jpg	36115	170	Sous-segmentation extrême : quasi absence de pixels prédits alors que le masque réel est riche.
20180630_154549.jpg	39378	472	Sous-segmentation : la prédiction est très inférieure au masque réel.
20180630_154609.jpg	40022	1223	Sous-segmentation : majorité des zones annotées manquées.
20180630_154617.jpg	31440	0	Aucun pixel détecté : prédiction totalement vide, échec complet.

- Sous-segmentation (under-segmentation) :

Le modèle ne prédit qu'une partie de la zone réelle.

Conséquences :

- Trop peu de pixels positifs prédits.
- Le masque prédit est plus petit que le masque réel.
- IoU et Dice sont faibles.

- Sur-segmentation (over-segmentation):

Le modèle prédit une zone plus grande que la réalité.

Conséquences :

- Trop de pixels positifs prédits.
- Le masque déborde de la vraie zone.
- IoU et Dice chutent si le chevauchement est faible.

IV.1.9 Problèmes rencontrés :

Lors de la comparaison entre les deux masques on trouve le résultat suivant :



17 masques prédits sauvegardés dans :
C:\Users\PC\Desktop\2C\projet_teledection\masques_predits_nouvelles

[20180630_154039.jpg](#): IoU=0.000, Dice=0.000
[20180630_154139.jpg](#): IoU=0.000, Dice=0.000
[20180630_154201.jpg](#): IoU=0.000, Dice=0.000
[20180630_154239.jpg](#): IoU=0.000, Dice=0.000
[20180630_154245.jpg](#): IoU=0.000, Dice=0.000
[20180630_154305.jpg](#): IoU=0.000, Dice=0.000
[20180630_154327.jpg](#): IoU=0.000, Dice=0.000
[20180630_154451.jpg](#): IoU=0.000, Dice=0.000
[20180630_154509.jpg](#): IoU=0.000, Dice=0.000
[20180630_154513.jpg](#): IoU=0.000, Dice=0.000
[20180630_154519.jpg](#): IoU=0.000, Dice=0.000
[20180630_154523.jpg](#): IoU=0.000, Dice=0.000
[20180630_154531.jpg](#): IoU=0.000, Dice=0.000
[20180630_154541.jpg](#): IoU=0.000, Dice=0.000
[20180630_154549.jpg](#): IoU=0.000, Dice=0.000
[20180630_154609.jpg](#): IoU=0.000, Dice=0.000
[20180630_154617.jpg](#): IoU=0.000, Dice=0.000

~~IoU~~ moyen : 0.000 (± 0.000)
~~Dice~~ moyen : 0.000 (± 0.000)

Résultats sauvegardés dans
C:\Users\PC\Desktop\2C\projet_teledection\masques_predits_nouvelles\resultats_comparaison.csv
et .xlsx

Analyse :

- Tous les scores sont à zéro c'est-à-dire aucune intersection entre masques prédits et réels.

- Les masques sont vides (sans annotations).
- Pour résoudre ce problème du masque réel on a suivi l'approche qu'on a utilisé pour annotés les masques dans la phase d'entraînement (voir la procédure ci-dessus).

Les masques ne sont plus donc vides ce qui nous a permet de faire les comparaisons voulues.

IV.2 Conclusion :

- **Le modèle donne des résultats inégaux** : il peut réussir sur certaines images mais échoue sur la plupart.
- **Axes d'amélioration possibles** : augmenter la quantité et la diversité des données d'entraînement, améliorer l'annotation, ou tester d'autres architectures de modèles.